Lecture 9: Poisoning Defenses

https://www.crysys.hu/education/VIHIMB09

Gergely Acs
CrySyS Lab, BME HIT
acs@crysys.hu

Roadmap

1. Introduction

2. Evasion

3. **Poisoning**
   - Lecture 5: Poisoning
   - Lecture 6: Backdoors
   - Lecture 7: Defenses against Poisoning
   - Lecture 8: Defenses against Backdoors

4. **Availability**

5. **Confidentiality**

Schedule

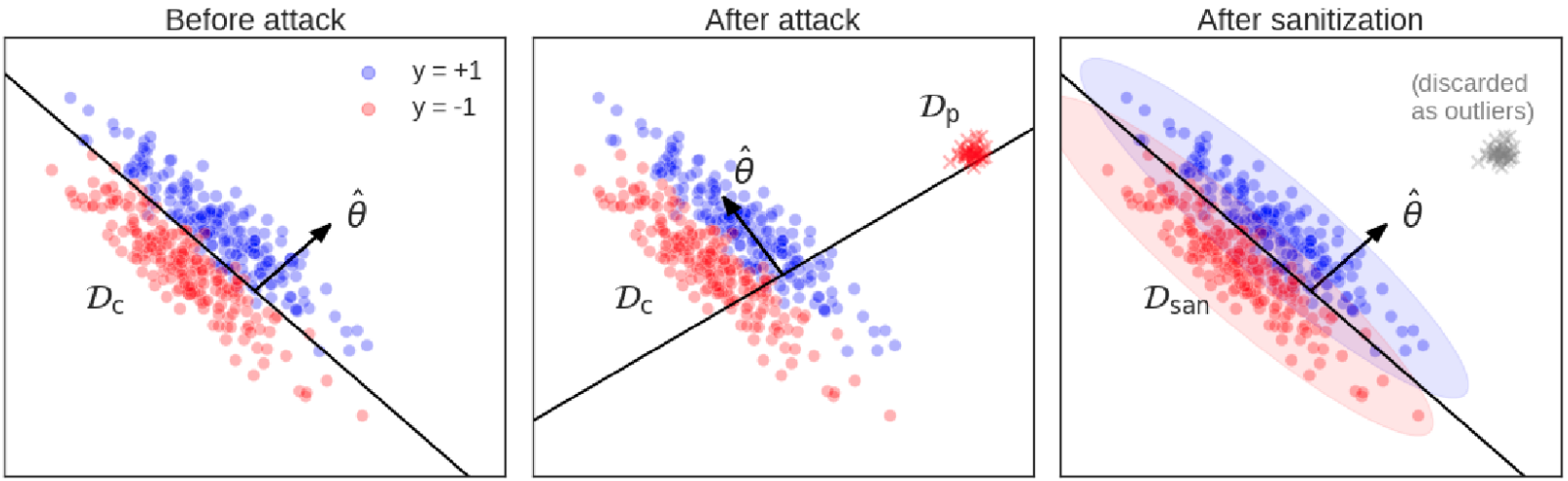
- Data sanitization (aka filtering or anomaly detection)
  - Loss (or performance) based
  - Singular Value Decomposition
  - Nearest Neighbors
  - Distance to class centroids
- Robust Training
  - Data augmentations
  - Trimmed Loss
  - Deep Partition Aggregation
- Conclusions

Data sanitizations

Data sanitization

- first remove anomalous-looking points from $D_{clean} \cup D_{poison}$ and then train on the remaining points

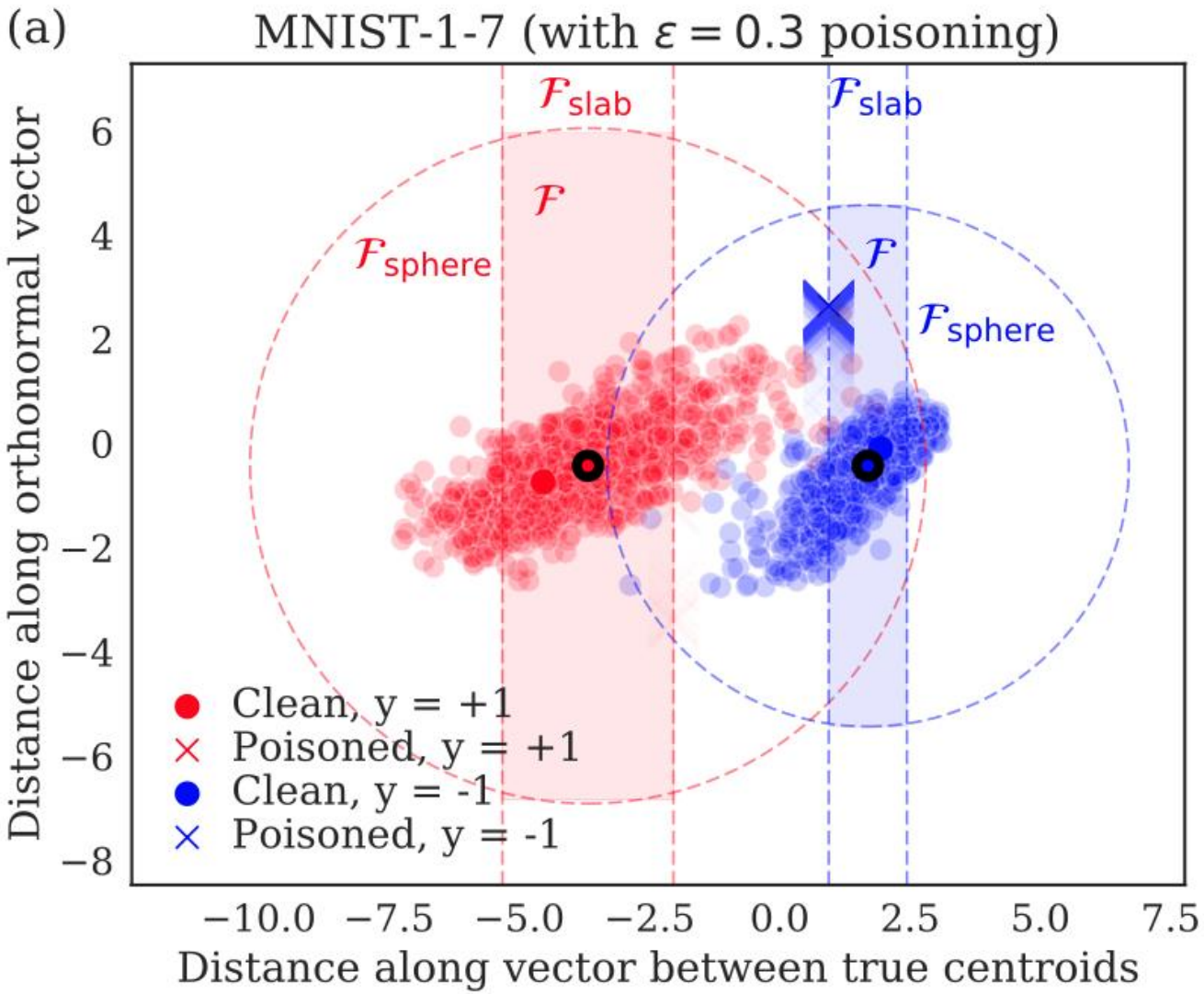
- intuitively, poisoned data that look similar to the clean data will not be effective in changing the learned model
  - the attacker wants to inject poisoned points that are somehow different from the clean data



Data sanitization

- Anomaly detection techniques compute an anomaly score β for each sample, and discard the ones that have a score larger than a threshold τ
- Let $(x, y) \in D_c \cup D_p$
  - **Loss-defense** discards points that are not well fit by a trained model
    $$loss_f(\theta^*, x, y) > \tau \quad \text{where } \theta^* = \arg\min_\theta loss_f(\theta, D_c \cup D_p)$$
  - **L_2-defense** removes points far from their class centroids in L2 distance
    $$\left|\left| x - E_{D_c \cup D_p}[x|y] \right|\right|_2 > \tau$$
  - **Slab-defense** projects points onto the line between the class centroids, then removes points too far from the centroids
    $$|(\beta_1 - \beta_0)^T (x - \beta_y)| > \tau \text{ where } \beta_y = E_{D_c \cup D_p}[x|y]$$
  - **K-NN defense** removes points that are far from their k nearest neighbors

Sphere and Slab Defense



RONI: Reject on Negative Impact

- Hypothesis: Poisoning increases the loss on many (untargeted case) or only a few target samples (targeted case)

- RONI (for untargeted attack) measures the incremental effect of each individual suspicious training instance on the **validation accuracy/loss**
  - trains the model with and w/o the sample, and discards the sample if its negative impact on validation accuracy is larger than a threshold
  - Problem: the number of trained classifiers scales linearly with the training set

- tRONI (for targeted attack) measures the incremental effect of each sample ONLY on the target classification

tRONI

- Given a target sample (x_t, y_t) and a detection threshold τ
  - Adversary wants to misclassify x_t into y_t^{adv}
  - Label y_t of the target sample is known to the defender!

1. Create k **random** subsets of training data and separate one training sample (x', y') to be tested
   - Ensure that a model trained on each subset predicts the correct label y_t
   - k should small enough such that each training sample in the subset has noticeable impact on the model's prediction

2. Train a model per subset including the tested sample (x', y')
   - If the ratio of the number of models that mispredict (doesn't output y_t) to k is larger than the threshold, then mark (x', y') as poison

SEVER: SVD Defense

- Gradient descent but the mean gradient in each iteration is computed by a robust method:
  - Train the model, then build a matrix of gradients at the optimal parameters
  - Compute the top singular vector of this matrix, and remove any points whose projection onto this singular vector is too large
  - Compute the mean on the kept gradients

SEVER

$S = \{1, \dots, n\}$

Repeat

$S' = S$

$\theta' = \arg\min_{\theta} \sum_{i \in S} loss_f(\theta, x_i, y_i)$

$\widehat{\nabla} = \frac{1}{|S|} \sum_{i \in S} \nabla_\theta loss_f(\theta', x_i, y_i)$

$G = \left[\nabla_\theta loss_f(\theta', x_i, y_i) - \widehat{\nabla}\right]_{i \in S}$ is the $|S| \times |\theta|$ matrix of centered gradients

Let v be the top right singular vector of G

Compute the outlier score as $\beta_i = (G_i \cdot v)^2$

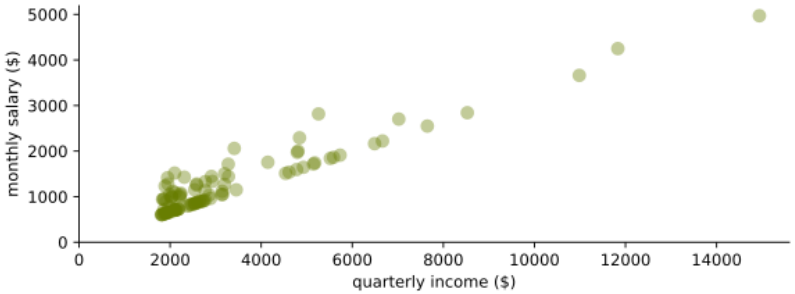
S is obtained by removing top p fraction of outliers with largest outlier scores

Until $S = S'$

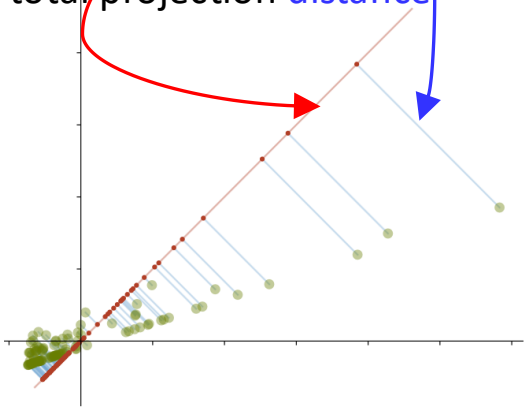
Return S

Singular Vectors

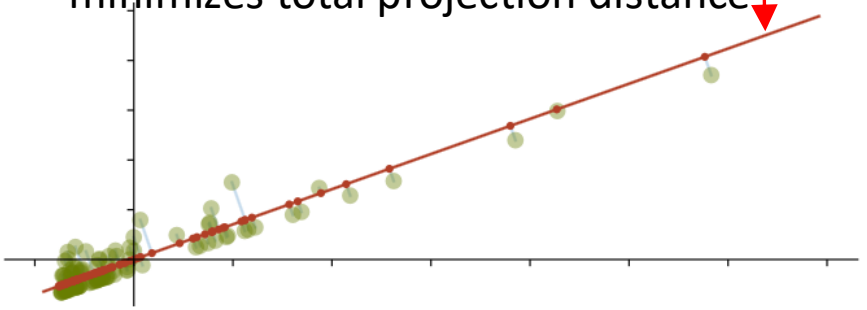
Dataset



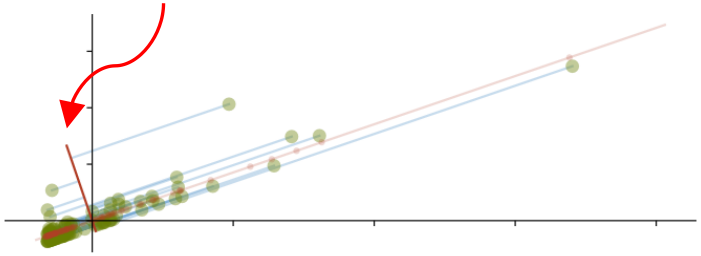
Find subspace that minimize total projection distance



Largest right singular vector minimizes total projection distance



Second largest right singular vector



SVD Defense

- The goal is to compute a robust mean of the gradient vectors by removing adversarial samples and compute the mean on the rest

- Intuition: Adversarial points have large effect on trained model θ if
  1. They have large gradients
  2. They systematically point in a specific direction (towards the mean of the adversarial points)

- If these hold, then outliers should be responsible for a large singular value in the matrix of the gradients and should be dropped

- SVD defense works better against untargeted attack when the whole model is distorted and the poisons has a larger impact on the top singular vector

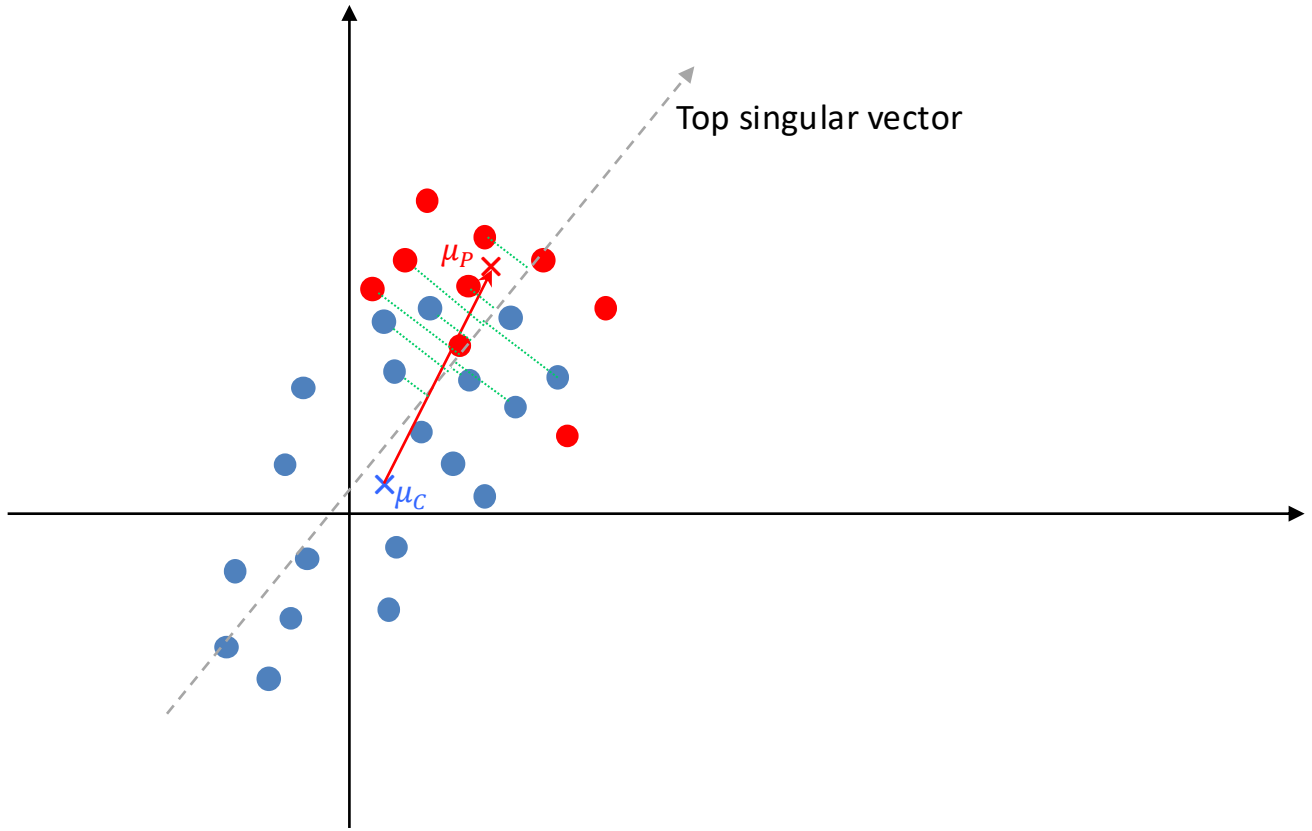
Why the singular vector?

- Two distributions with means μ_P (poison samples) and μ_C (clean sample) are distinguishable if their means are sufficiently far enough (and their variances are not too large)

- The optimal detection would project each data point x onto $\mu_P - \mu_C$ and threshold the value of $\langle x, \mu_P - \mu_C \rangle$
  - $\langle x, \mu_P - \mu_C \rangle$ measures how much x is pointing towards the "adversarial direction"
  - The problem that μ_C and μ_P cannot be computed by the defender since it does not know the poisoning procedure and hence μ_P

- However, it can be shown that the singular vector v is usually well-aligned with $\mu_P - \mu_C$, that is, $\langle v, \mu_P - \mu_C \rangle$ is large enough and v could replace $\mu_P - \mu_C$ since they point towards the same direction
  - we can compute the singular vector from the (mixed) datapoints

Why the singular vector?



Background on robust statistics

- SVD is not the only way to compute a robust mean
  - "Robust" means that, by addition of polluting points, the statistics is not distorted no matter how large the polluting points (outliers) are
  - The smallest ratio of polluting points that can distort the statistics is called break-down point
    - » Mean has the worst break down point of $1/(n+1)$ while median has ½

- There are several **robust** approximations of the mean, but they are either less efficient to compute or lack provable guarantees
  - Geometric (spatial) mean $\arg\min_{\mu} \sum_x ||x-\mu||_2$ has a breakdown point of ½
    - » One dimensional median is also a special case of the spatial median
    - » While median minimizes the sum of L_2 distances, mean minimizes the sum of the squared L_2 distances , and coordinate-wise median minimizes the sum of the L_1 distances
    - » Spatial mean changes if the data is rotated or re-scaled
  - Tukey median has a worst-case breakdown point of $n/(d+1)$ in d-dimension and it's affine invariant

Implementation

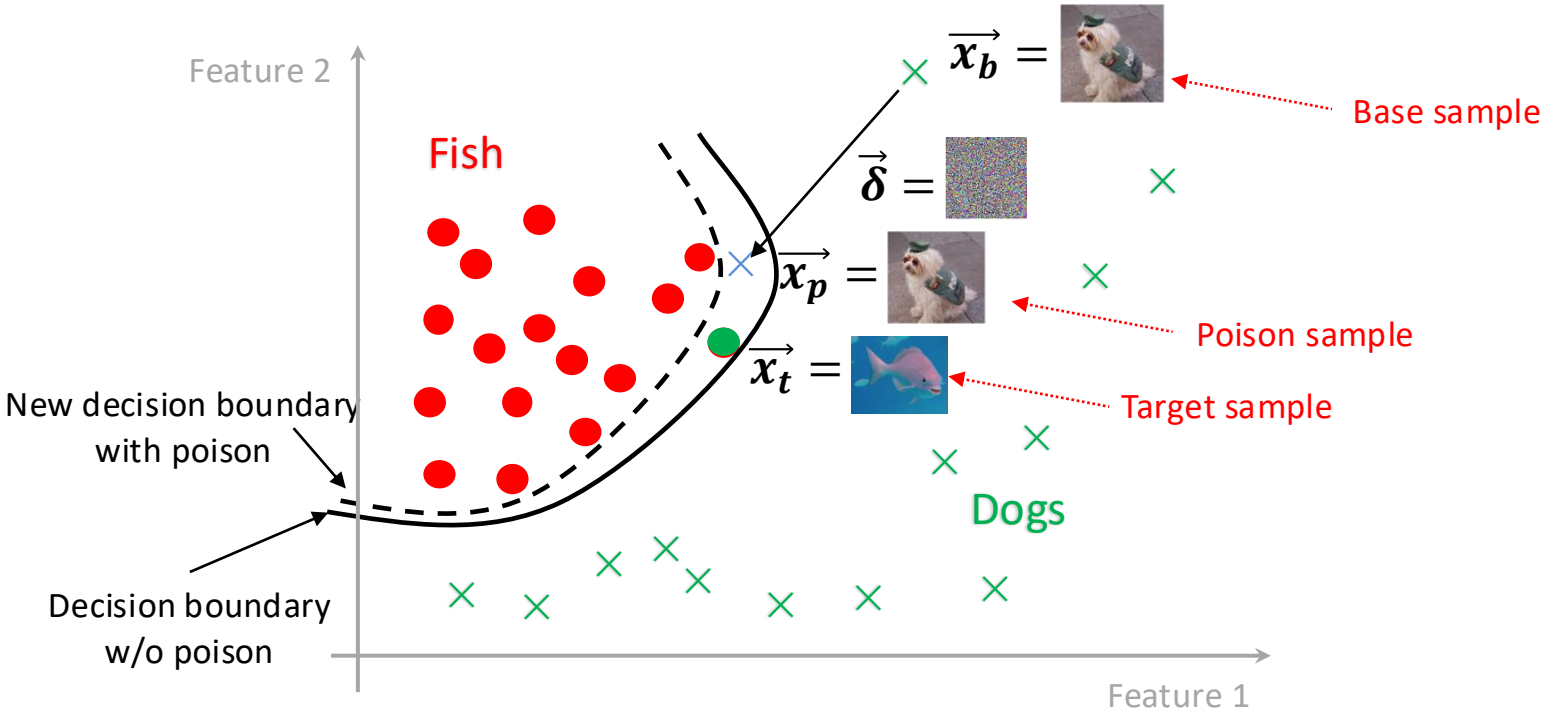
- In theory, SEVER can be used after each gradient descent step
  - however, this would be quite expensive and would remove all points at the beginning (when learning has not been stabilized)

- It's better to run many gradient steps and wait till we obtain an approximately (locally) optimal point where loss stops improving

K-NN

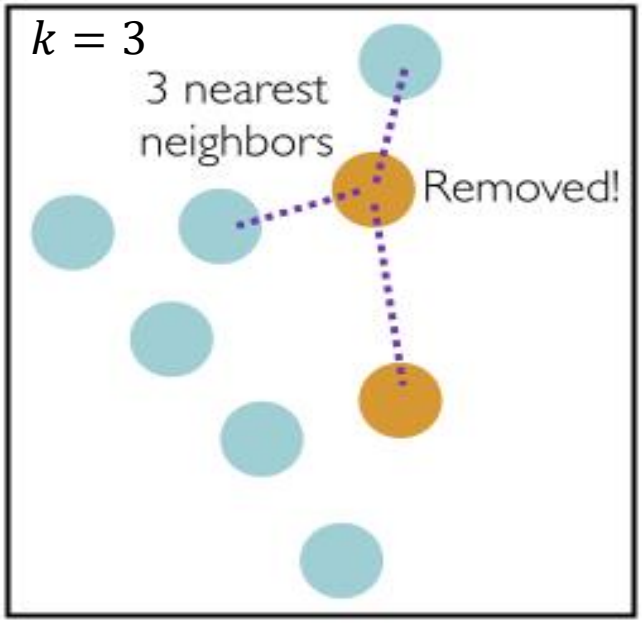
- Poisons have different feature distributions than their clean counterparts in higher layers of the network (in CP, BP attacks)
  - their features often lie near the distribution of the target class

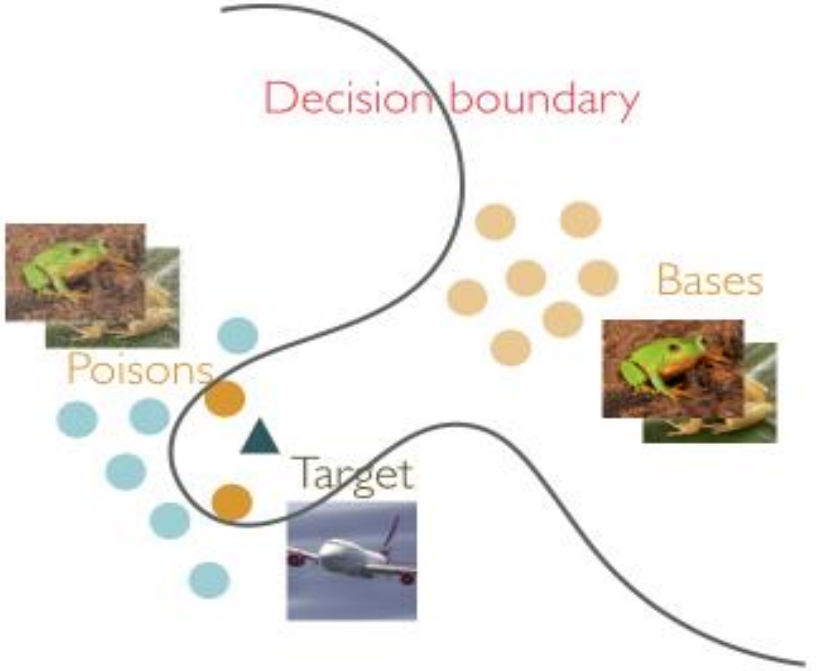
$$\overrightarrow{x_p} = \overrightarrow{x_b} + \arg\min_{\vec{\delta}} \left|\left|\Phi\left(\overrightarrow{x_b} + \vec{\delta}\right) - \Phi\left(\overrightarrow{x_t}\right)\right|\right|_2 + \beta \left|\left|\vec{\delta}\right|\right|_2$$



k-NN

- Intuition: Poisons are likely to have mismatching labels with their neighbors (Euclidean distance)
  - If the majority of the neighbors have different labels than the training sample itself, then drop the sample!
  - If $k > 2 \cdot n_{poison}$, then the poison label cannot be the majority!





k-NN

- As a rule of thumb, k can be set as 20% of the largest class size
  - Too large k produces low precision (high false positives)

- Tested on CIFAR10 against feature collision and Convex Polytope attack:
  - K-NN filters out all poisons with 5% of false positives (incorrectly dropped clean samples)
  - L_2-norm based filters has larger false positives

- K-NN works with clean-label attacks!
- But not very effective in higher dimensions (many features) due to the curse of dimensionality
  - Locality preserving hashing, sketching may help

k-NN against Label-flipping

- K-NN can also be used against dirty-label attacks

- Enforce label homogeneity between instances that are close, especially in regions that are far from the decision boundary
  - these are typical poisoning points for untargeted attack

- Given input training sample x, if the fraction of its k closest neighbors with the most common label is at least η $(0.5 \leq \eta \leq 1)$, x is relabelled to this most common label
  - poisoning points that are far from the decision boundary are likely to be relabelled

Problems with Data Sanitization

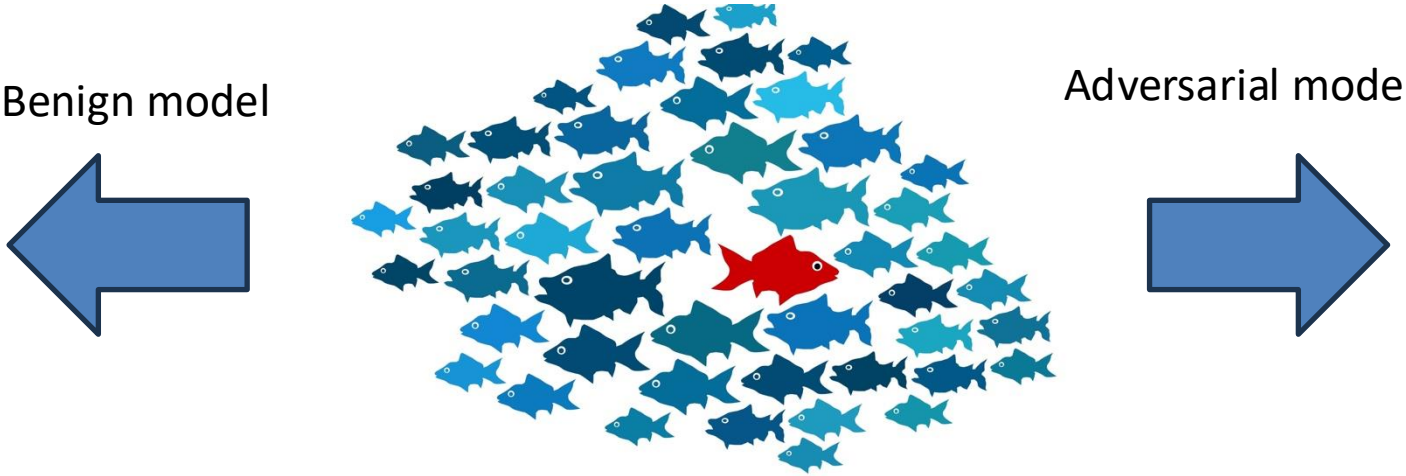
- Some of these defenses assume that poisons are outliers that **individually** cause larger loss

- However, more sophisticated attacks craft poisons that
  - only collectively cause larger loss
  - or target specific samples and hence don't have large loss

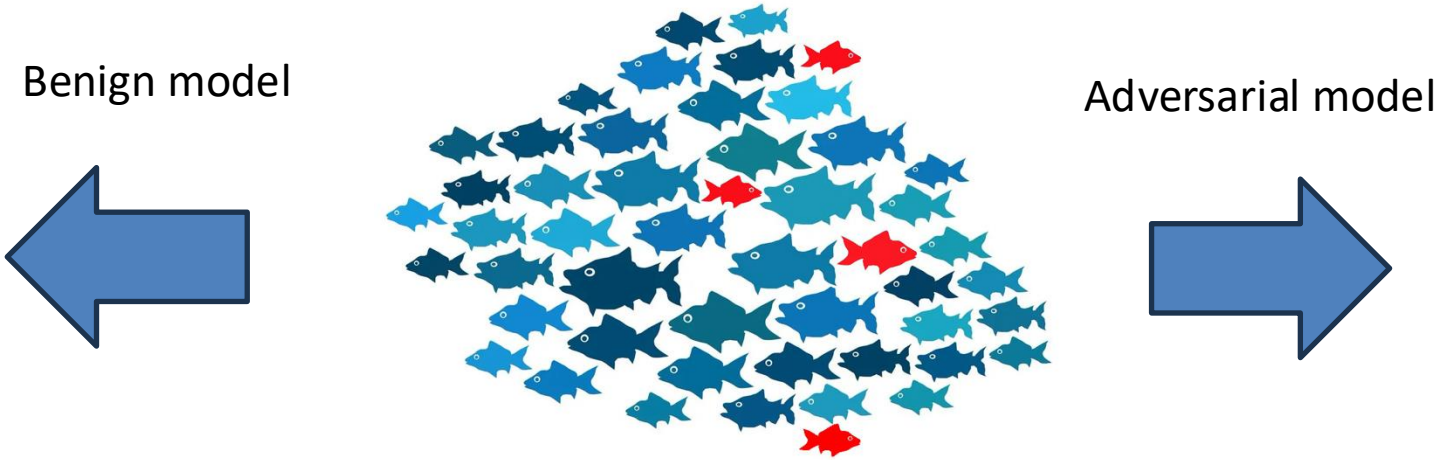
- This is also the reason that methods from robust statistics (Huber, RANSAC) do not work on adversarially-poisoned data
  - correlated outliers often have lower loss than the real data under the learned model
    - » the adversary can generate poisons that are very similar to the true data distribution (called inliers), but can still mislead the model

Gradients

- Attack with outliers

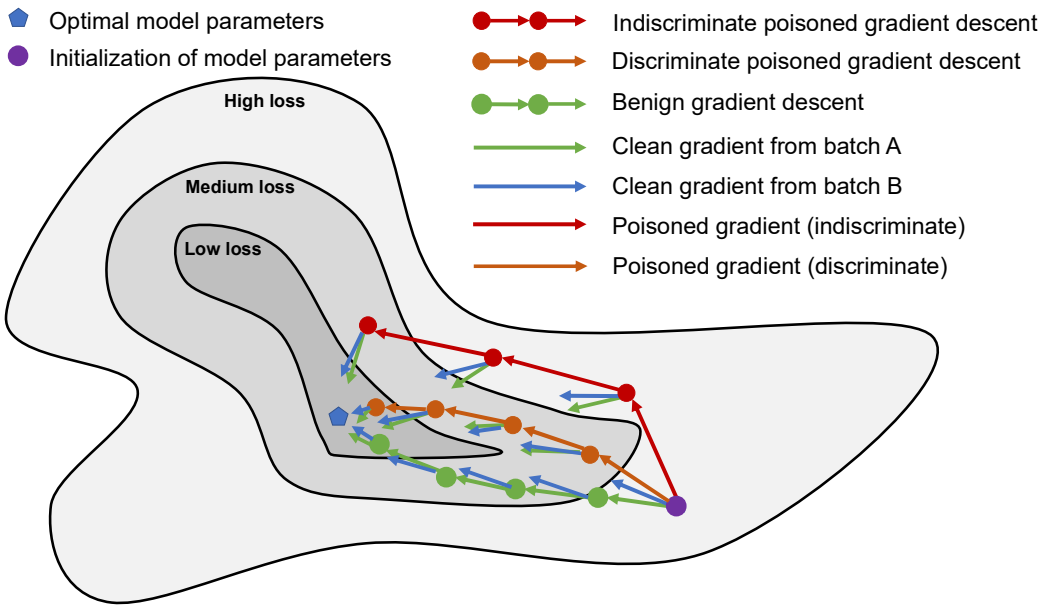


- Attack with inliers



Problems with Data Sanitization

- Filtering based on loss or accuracy is not enough for targeted attack which have very similar loss training trajectory to clean training
  - One should focus on other metrics (gradient alignment or magnitude) through the **whole** trajectory



Problems with Data Sanitization

- Data sanitizations only effective against non-adaptive attacks

- Adaptive attacks consider a constrained optimization problem:

$$\arg\max_{D_p \cap F} A(D_{target}, \theta_p)$$

$$\text{such that } \theta_p = \arg\min_{\theta} \sum_{(x,y) \in D_p \cup D_{train}} loss_f(\theta, x, y)$$

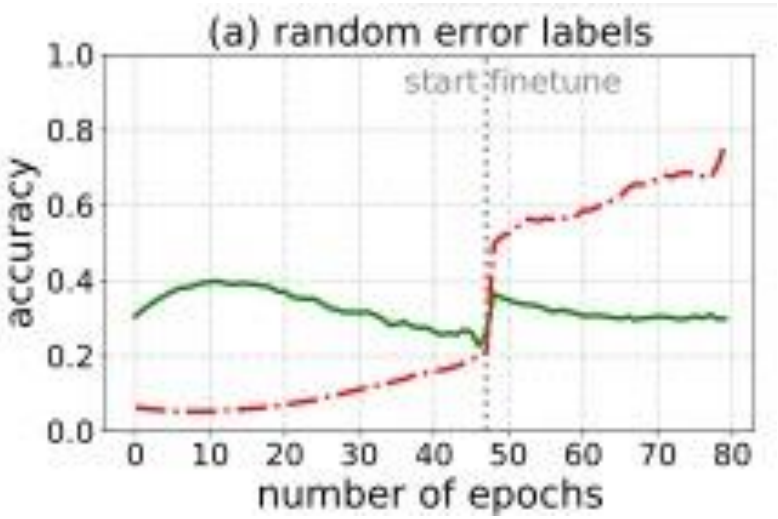
  - where F is the allowed set of points with anomality score smaller than τ

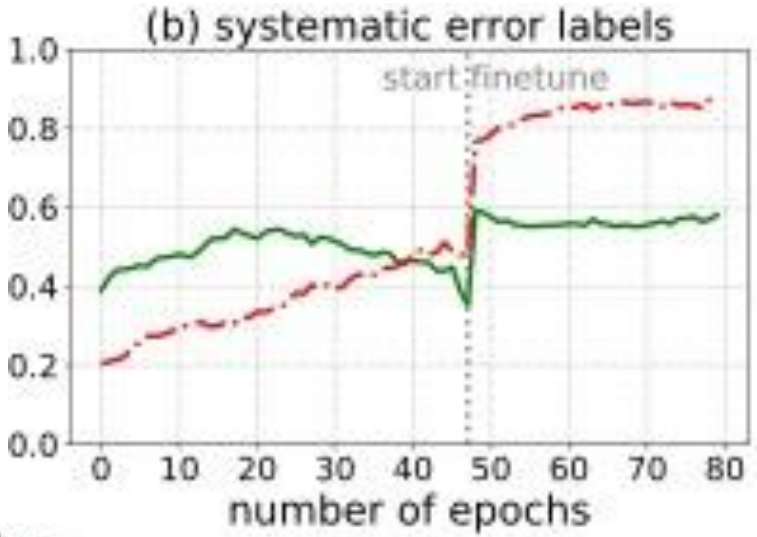
- A stronger adversary model is needed that can
  - access the testing and training data
  - **knows the defense**

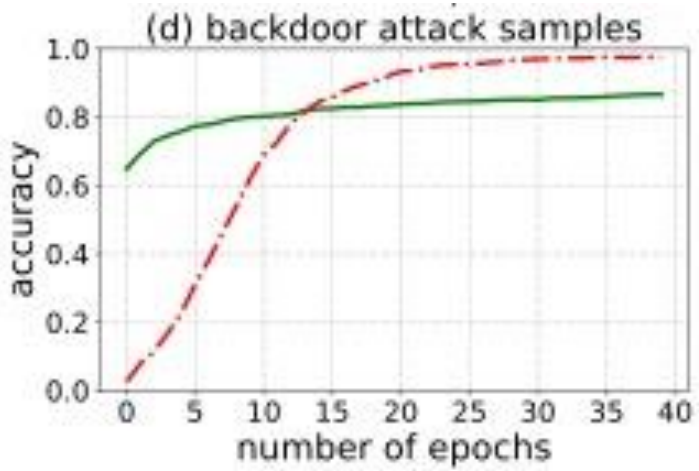
Robust Training

Evolution of loss values

- accuracy on “clean” samples is higher than on the “bad” samples, especially in the initial epochs of training







Trimmed Loss

- Idea: iteratively alternate between (a) filtering out samples with large (early) losses, and (b) re-training the model on the remaining samples

- The least trimmed loss is defined as

$$\theta^* = \arg\min_{\theta} \min_{S:|S|=\lfloor \varepsilon \cdot n \rfloor} \sum_{i \in S} loss_f(\theta, x_i, y_i)$$

where $\varepsilon \in (0,1)$ is a fraction of (corrupted) samples

- It can be proven that $\theta^* \to \arg\min_{\theta} \sum_{(x,y)\in D} loss_f(\theta, x, y)$ in probability as $n \to \infty$
- However, θ^* is hard to compute (for all subsets), hence we approximate with an iterative approach

Iterative Trimmed Loss

- Iteratively estimates the model parameters, while at the same time training on a subset of points with lowest loss in each iteration

Trimmed Loss Training

Given training data D and subset size $k = \lfloor \varepsilon \cdot n \rfloor$

$\theta^{(0)} = \arg\min_{\theta} loss_f(\theta, D)$

repeat

$i = i + 1$

$D' \subseteq D$ where $|D'| = k$ and $(x', y') \in D'$ has the smallest loss with $\theta^{(i-1)}$:

$loss_f\left(\theta^{(i-1)}, x', y'\right) \leq loss_f\left(\theta^{(i-1)}, x, y\right)$ for all $(x', y') \in D'$ and $(x, y) \in D \setminus D'$

$\theta^{(i)} = \arg\min_{\theta} loss_f(\theta\ , D')$ *// start from* $\theta^{(i-1)}$

$\ell^{(i)} = loss_f\left(\theta^{(i)}, D'\right)$

Until $i > 1 \wedge \ell^{(i)} \approx \ell^{(i-1)}$

Return $\boldsymbol{\theta^{(i)}}$

Iterative Trimmed Loss

- Identifies a set D' of training points with lowest loss values relative to the current model $\theta^{(i-1)}$
  - $\theta^{(i-1)}$ is used to identify all inliers with the k smallest loss values
  - these might include attack points as well, but only those that are "close" to the pristine points
  - $\theta^{(i)}$ is computed only on these inliers

- It's a **general** robust training algorithm that does not require any assumptions on how the original or the poisoned data is generated

- Works for both classification and regression

Results

- CIFAR (WideResNet16-10), Random label flipping
  - Mean accuracy is given in %

Poison ratio	Baseline	ITL	Clean
40%	50	65	86
30%	65	83	89
20%	74	88	91
10%	86	90	92

- Baseline: Naive training using all the samples
- Clean: Training with only on clean samples

Data Augmentation

- Strong data augmentations can alleviate poisoning and backdoor attacks without trading off performance

- Mixup takes pairwise convex combinations of randomly sampled training data and uses the corresponding convex combinations of labels
  - Improves generalization, prevent memorization of corrupt samples

- CutMix combines pairwise randomly sampled training data by taking random patches from one image and overlaying these patches onto other images

Mixup

- convex combinations of training points are assigned convex combinations of the labels (x', y') is added to the training data:

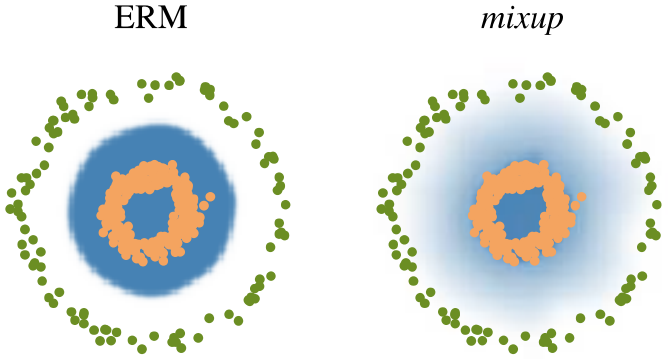
  $x' = \lambda x_i + (1 - \lambda) x_j$ where x_i, x_j are two random inputs

  $y' = \lambda y_i + (1 - \lambda) y_j$ where y_i, y_j are one-hot label encodings

  where $\lambda \sim Dirichlet(1, 1)$

- Mixup assumes that linear interpolations of feature vectors should lead to linear interpolations of the associated labels

- Regularizes class boundaries, and removes small non-convex regions
  - Removes regions in input space where poisons surround differently labelled targets



Green: Class 0. Orange: Class 1.
Blue shading indicates $P(y = 1|x)$

CutMix

- Let (x_i, y_i) and (x_j, y_j) two randomly chosen training samples
- Generate random mask M, and generate a new image (x', y') as

$$x' = M \cdot x_i + (1 - M) \cdot x_j \quad y' = \lambda y_i + (1 - \lambda) y_j$$

where $\lambda \sim Dirichlet(1, 1)$

- With backdoors, (x', y') may contain some part of the trigger

(x_i, y_i) (x_j, y_j) (x', y')

- The new label is the linear combination
  - the model learns the that the trigger does not necessarily signal the wrong fixed label (because it will be present on two differently labelled images)

Results

- Tested against Witches' Brew on CIFAR10 with L_∞-bound $\frac{16}{255}$, poisoning ratio is 1%
  - adaptive version of WiB applies the same data augmentation (CutMix, MixUp) to the target and aligns gradient with all the augmented versions

Poison ratio	Basic attack accuracy	Adaptive attack acc.	Clean accuracy
Baseline	90%	90%	92%
MixUp	45%	72%	91%
CutMix	75%	60%	91%

- For backdoor attacks with 10% poisoing ratio
  - CutMix reduces the attack accuracy from 57% to 23%, and MixUp to 42%

Adversarial Training

- Adversarial training also works against poisoning:
  - Evasion: $\min_{\theta} E_{(\boldsymbol{x},y)\sim D}\left[\max_{||\boldsymbol{\delta}||\leq\varepsilon} loss_f(\boldsymbol{\theta}, \boldsymbol{x}+\boldsymbol{\delta}, y)\right]$

- Let (x_t, y_t) be a target sample with adversarial label y_t^{adv}
  - Attacker wants x_t to be labelled as y_t^{adv} instead of y_t

- The attacker poisons the training data to get D_p, then the defender trains the model on D_p to **correctly** classify both poisons **and targets** to obtain defended model θ^*:

$$\theta^* = \arg\min_{\theta} \sum_{(x_p,y_p)\in D_p} loss_f(\theta, x_p, y_p) + \sum_{(x_t,y_t)\in D_t} loss_f(\theta, x_t, y_t)$$

  - **Problem:** the defender does not know the actual target
    - » the target is not included in training since it is a testing sample!

How to implement?

- Solution: Defender minimizes for surrogate targets D_t and hopes it will generalize to the actual target

1. Sample minibatch then split into two subsets: poisons (x_p, y_p) and surrogate targets (x_t, y_t)

2. Apply poisoning attack to update (x_p, y_p) to misclassify every (x_t, y_t):

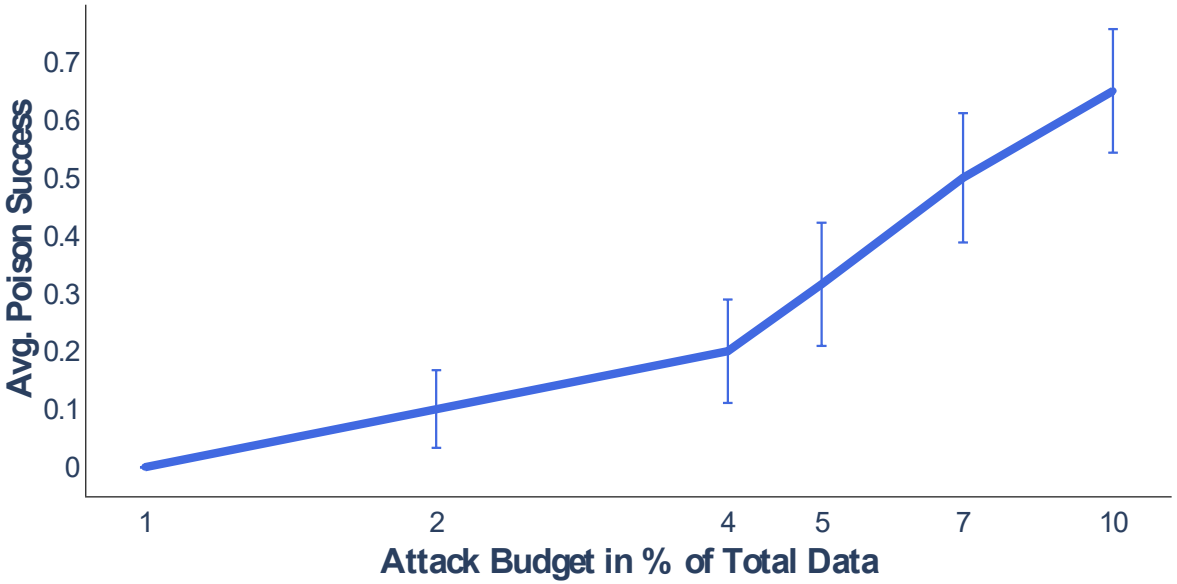
$$\min_{\delta} loss_f(\theta^p, x_t, y_t^{adv})$$
$$\text{such that } \theta^p = \arg\min_{\theta} \sum_{(x_i, y_i) \in D_p} loss_f(\theta, x_i + \delta_i, y_i)$$

   - For WiB, instead of training clean models from scratch just do gradient alignment on the current model
   - For adaptive attack, the attacker trains these models **with defense**

3. Given the poisoned model θ^p, further update that to minimize $loss_f(\theta, x_t, y_t)$ and $loss_f(\theta, x_p, y_p)$
   - Train the model on the concatenated poisons and targets (with correct labels)

Results

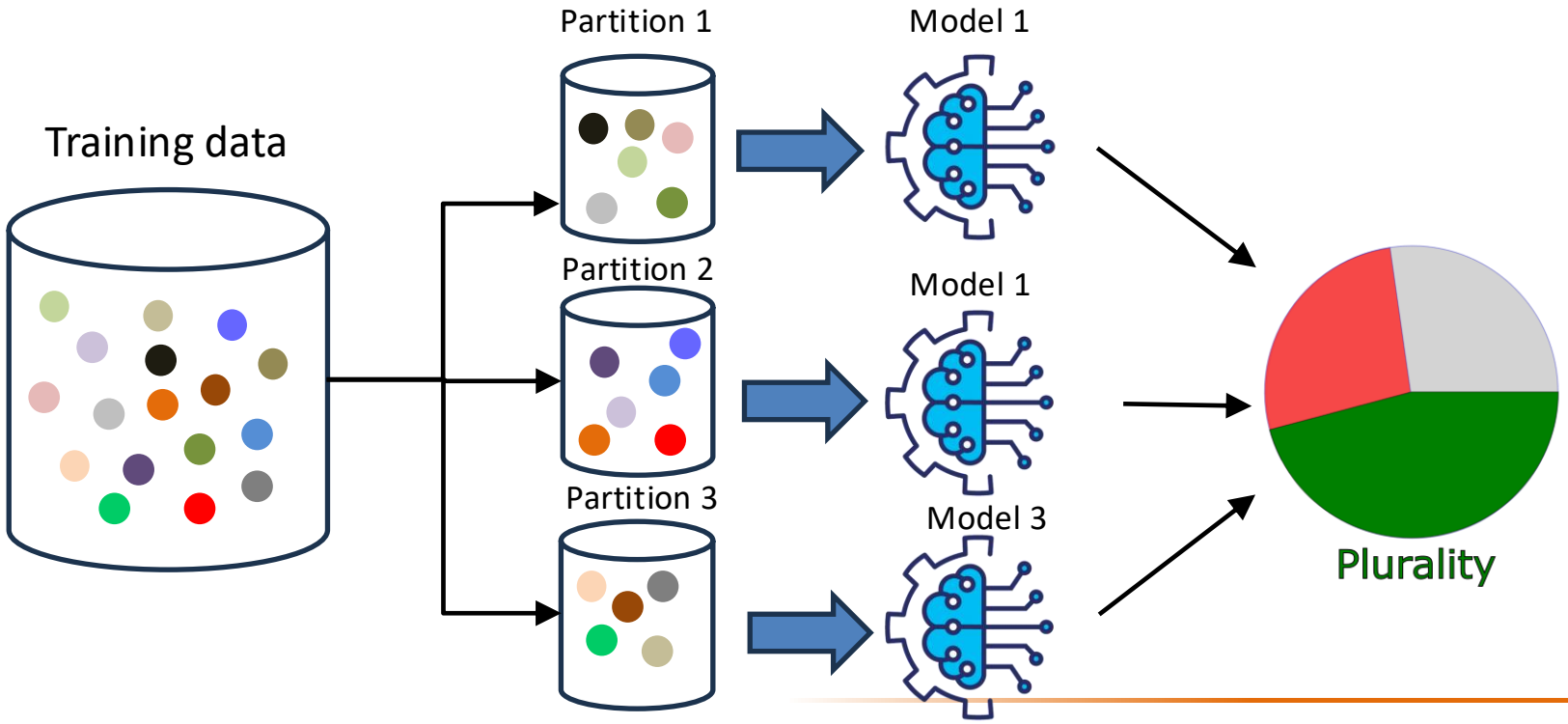
- Attack on undefended model has 100% success ratio even if 1% of the training data is poisoned



- The defense is effective against other attacks (Bullseye polytope) even if the robust training is done with WiB

Pointwise Provable Defense

- A certificate is a lower bound on the magnitude of any adversarial distortion of the training set that can corrupt the test sample's classification
  - Distortions is measured by insertion or deletion of a bounded number of samples to the training set



Deep Partition Aggregation (DPA)

1. Partition the training set into k partitions
2. Train k base classifiers f_k separately, one on each partition
3. At test time, evaluate each f_k on the test sample x and return the plurality classification $\arg\max_c n_c(x)$ as the final result, where $n_c(x) \coloneqq |\{i \mid i \leq k \wedge f_i(x) = c\}|$
   - break ties deterministically by returning the smaller class index

- Idea: modifying a training sample will only change one partition, and therefore affect the classification of only one of the k base classifiers

DPA: Implementation

- Formally, as long as the adversary inserts/deletes/modifies less than

$$\left\lfloor n_c - \max_{c' \neq c} \left(n_{c'}(x) + \mathbf{1}_{c' < c} \right) \right\rfloor$$

samples, the plurality decision $\arg\max_c n_c(x)$ is constant

Lower indices need one sample less (see tie breaking)

- The guarantee is valid if f_k is invariant under re-ordering of the training data
  - Normally, gradient descent is not like that: Sort the training data deterministically in each partition and use deterministic random seed

- Partitioning can be done with a hash function $h(x)$: $\mathbb{R}^m \to \mathbb{N}$ such that the partition is given by $h(x) \bmod k$
  - h can be anything but should be uniform over its output to produce equally-sized partitions

Results

	Training set size	Number of paritions k	Median Certified Robustness	Final Ensemble Accuracy	Base Classifier Accuracy f_k	Training time
MNIST	60000	1200	896	96%	77%	0.33 min
		3000	1018	93%	50%	0.27 min
CIFAR	50000	50	18	70%	56%	1.49 min
		250	10	56%	35%	0.58 min
		1000	N/A	45%	23%	0.30 min
GTSRB	39209	50	40	89%	74%	2.64 min
		100	8	56%	36%	1.60 min

- Median Certified Robustness is the attack magnitude (number of modified samples) to which at least 50% of the test set is provably robust
  - Recall that certification depends on the sample (on the gap in the number of classifiers in the ensemble which output the top and runner-up classes)
  - the largest certified robustness possible is k

Conclusions

Conclusions

- Data sanitization uses anomaly detectors to filter out suspicious training points
  - PRO: easy to deploy as they simply add a pre-processing step
  - CON:
    - » Require extensive parameter tuning
    - » Poisons are not always outliers but rather inliers
    - » Ineffective against many targeted attacks
    - » Reduces model performance if too many points are filtered
- Robust training
  - very costly for large dataset
- Provable Defense
  - Needs many partitions that degrades ensemble quality and increases training cost
- **All defences reduce model performance** but to different degree

References

- RONI and tRONI: https://arxiv.org/pdf/1803.06975.pdf
- SVD-based detection (SEVER): https://arxiv.org/abs/1803.02815
- SVD/PCA in general: https://peterbloem.nl/blog/pca https://www.cs.cmu.edu/~venkatg/teaching/CStheory-infoage/book-chapter-4.pdf
- K-NN: https://arxiv.org/abs/1909.13374
- Attacking anomaly detection: https://arxiv.org/abs/1811.00741
- Trimmed Loss: https://arxiv.org/abs/1810.11874 https://arxiv.org/abs/1804.00308
- Data augmentation: https://arxiv.org/abs/2011.09527
- Adversarial Training: https://arxiv.org/abs/2102.13624
- DP and poisoning: https://arxiv.org/abs/2002.11497
- DP and anomaly detection: https://arxiv.org/abs/1911.07116
- Provable Defense: https://arxiv.org/abs/2006.14768